CS-E4740 - Federated Learning '26

L5 - Federated Clustering

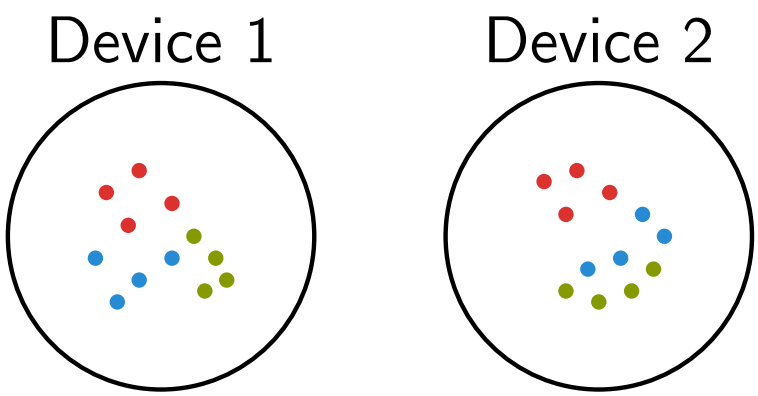
Salvatore Rastelli
Shamsiiat Abdurakhmanova

Spring 2026

Calendar **Glossary** **Book** **GitHub**

Federated clustering

Goal: Cluster the data points



Clustered federated learning (FL)

Goal: cluster the devices.

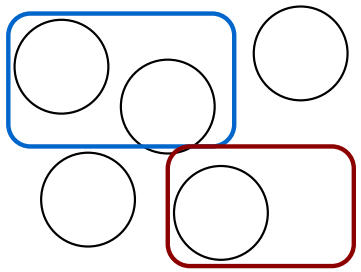


Table of Contents

1. Clustering
2. Hard clustering with k-means
3. Federated k-means
4. Soft clustering with Gaussian mixture models
5. Federated GMM

Definition

- **Clustering** aims to partition observations into groups (clusters) such that:
  - points within the same cluster are **similar**,
  - points in different clusters are **dissimilar**.
- Similarity is defined by a distance (or dissimilarity) measure

Hard clustering

Each observation is assigned to exactly one cluster (e.g., k-means).

Soft clustering

Each observation is associated with a probability distribution over clusters (e.g., GMMs via EM).

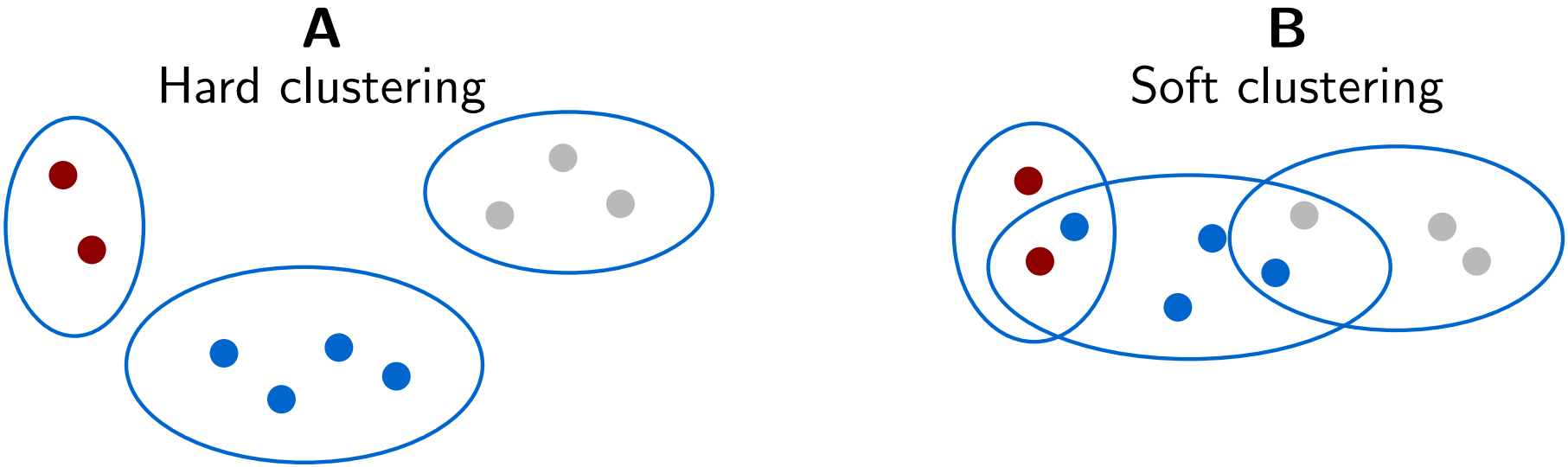


Table of Contents

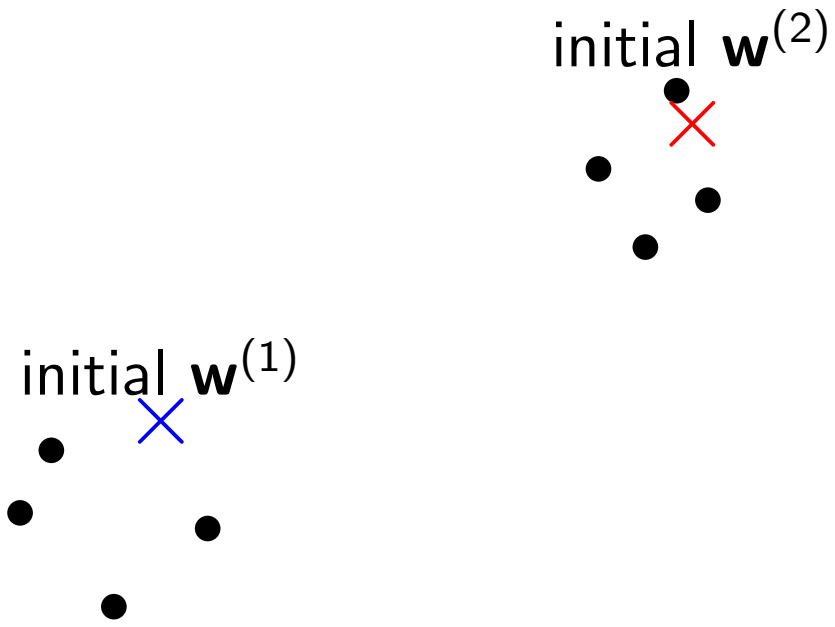
1. Clustering
2. Hard clustering with k-means
3. Federated k-means
4. Soft clustering with Gaussian mixture models
5. Federated GMM

What is k-means?

- k-means is a **hard clustering** algorithm.
- Each data point belongs to **exactly one cluster**.
- Each cluster represented by a cluster centroid.
- Goal: minimize cluster error, measured via euclidean distance between data points and their nearest cluster centroid.

k-means algorithm — Step 1: Initialization

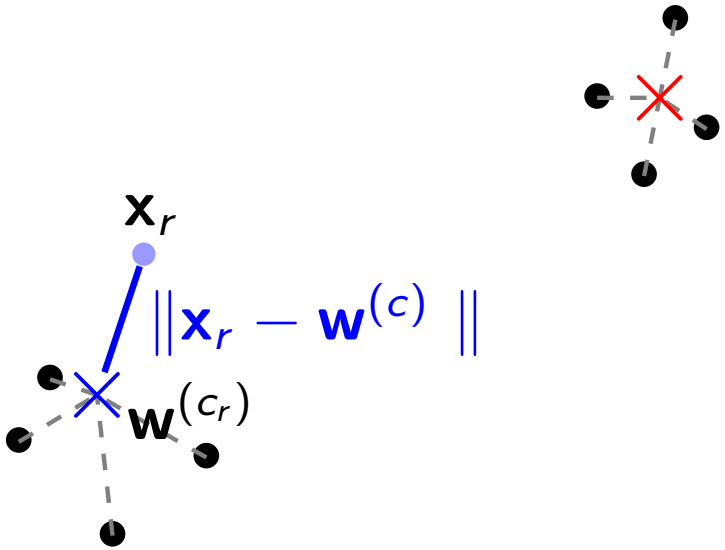
- Choose k initial cluster centroid
- Often selected **randomly from the data**
- Initialization affects convergence and solution quality



k-means algorithm — Step 2: Assignment

$$c_r = \arg\min_c \|\mathbf{x}_r - \mathbf{w}^{(c)}\|^2$$

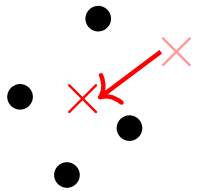
- Assign each point to the **nearest cluster centroid**
- This minimizes the objective for fixed cluster centroid

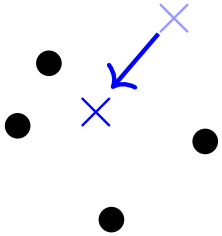


k-means algorithm — Step 3: Update

$$\mathbf{w}^{(c)} = \frac{1}{|\mathcal{C}_c|} \sum_{\mathbf{x}_r \in \mathcal{C}_c} \mathbf{x}_r$$

- Update each cluster centroid as the **mean of assigned points**
- Decreases the objective for fixed assignments
- Repeat Steps 2–3 until convergence





k-means algorithm

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(k)}\in\mathbb{R}^d} \underbrace{\sum_{\mathbf{x}\in\mathcal{D}} \min_{c\in[k]} \|\mathbf{x}-\mathbf{w}^{(c)}\|^2}_{\text{local } k\text{-means loss}}$$

1. Initialize cluster centroid $\{\mathbf{w}^{(c)}\}_{c=1}^{k}$
2. **Assignment step**:

$$c_r = \arg\min_c \|\mathbf{x}_r - \mathbf{w}^{(c)}\|^2$$

3. **Update step**:

$$\mathbf{w}^{(c)} = \frac{1}{|\mathcal{C}_c|} \sum_{\mathbf{x}_r\in\mathcal{C}_c} \mathbf{x}_r$$

4. Repeat until convergence

Table of Contents

1. Clustering
2. Hard clustering with k-means
3. Federated k-means
4. Soft clustering with Gaussian mixture models
5. Federated GMM

Motivation

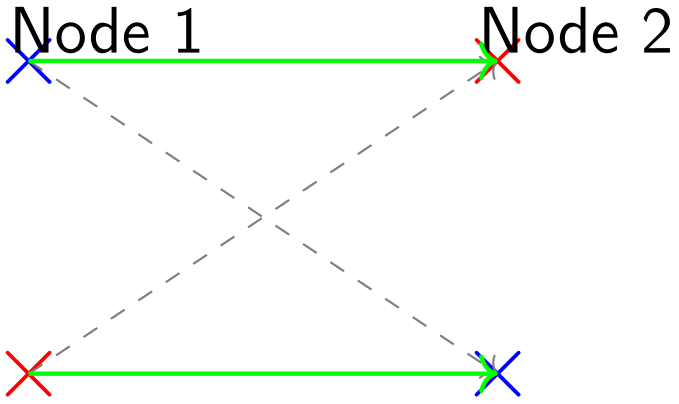
- devices collect **local datasets** $\mathcal{D}^{(i)}$
- Goal: obtain a **global clustering** of $\mathcal{D}^{(global)} = \bigcup_i \mathcal{D}^{(i)}$ under **privacy** and **communication** constraints.
- Challenge: each node has only a **limited amount of data**, which may not be sufficient for reliable clustering on its own.

Problem setup

- n devices (nodes) indexed by $i \in \{1, \ldots, n\}$ with local datasets $\mathcal{D}^{(i)}$.
- Communication over a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ using **neighbor links** $\mathcal{N}^{(i)}$.
- Each node maintains local cluster centroid $\mathbf{w}^{(i)} = \left(\mathbf{w}_1^{(i)}, \ldots, \mathbf{w}_k^{(i)}\right)$.
- Objective: learn cluster centroid without exchanging raw data.

Why naive aggregation fails

Issue: cluster labels are arbitrary at each node (permutation ambiguity).



- Directly averaging cluster centroid can **mix unrelated clusters**.
- We need a **permutation-invariant** way to measure cross-node disagreement.

GTVmin for federated k-means

- We formulate federated k-means as a **generalized total variation minimization (GTVMin)** problem:

$$\min_{\{\mathbf{w}^{(i)}\}_{i=1}^{n}} \sum_{i=1}^{n} \underbrace{\sum_{\mathbf{x}\in\mathcal{D}_i} \min_{c} \|\mathbf{x}-\mathbf{w}^{(i,c)}\|^2}_{\text{local } k\text{-means loss}} + \lambda \underbrace{\sum_{\{i,j\}\in\mathcal{E}} \underbrace{d^{(i,i')}}_{\textit{discrepancy}}}_{\text{graph GTV (GTV)}}$$

- Local term: fits private data at each node.
- GTV term: encourages **neighbor agreement** over the graph.

Discrepancy

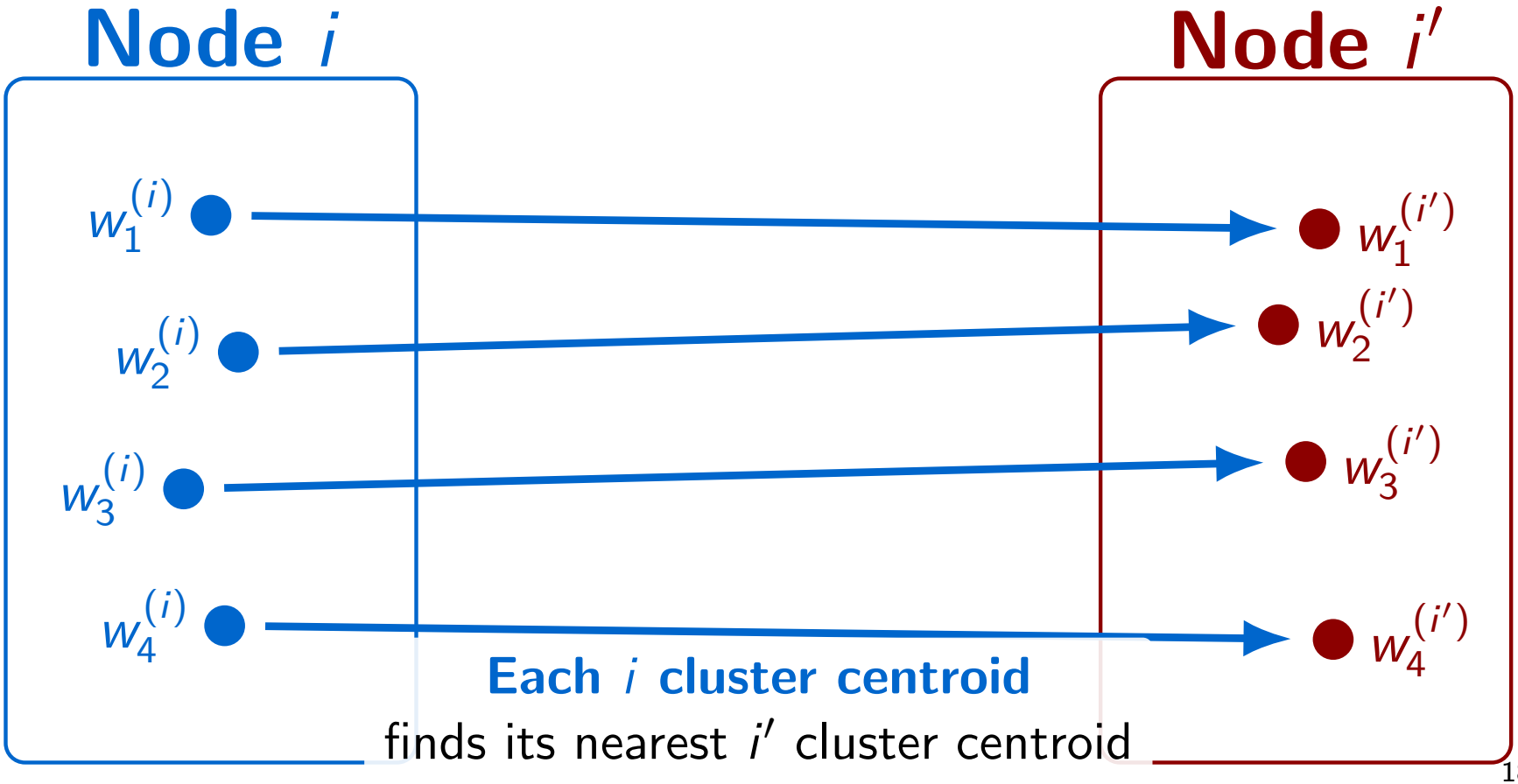
- Disagreement is measured between **sets of cluster centroid** (the order does not matter).
- The discrepancy between nodes i and i' is symmetric:

$$d^{(i,i')} = \underbrace{\sum_{c=1}^{k} \min_{c'} \|\mathbf{w}_c^{(i)} - \mathbf{w}_{c'}^{(i')}\|^2}_{\text{match } i \text{ cluster centroid to nearest in } i'} + \underbrace{\sum_{c=1}^{k} \min_{c'} \|\mathbf{w}_c^{(i')} - \mathbf{w}_{c'}^{(i)}\|^2}_{\text{match } i' \text{ cluster centroid to nearest in } i}$$

- Each term performs nearest-cluster centroid matching across nodes.

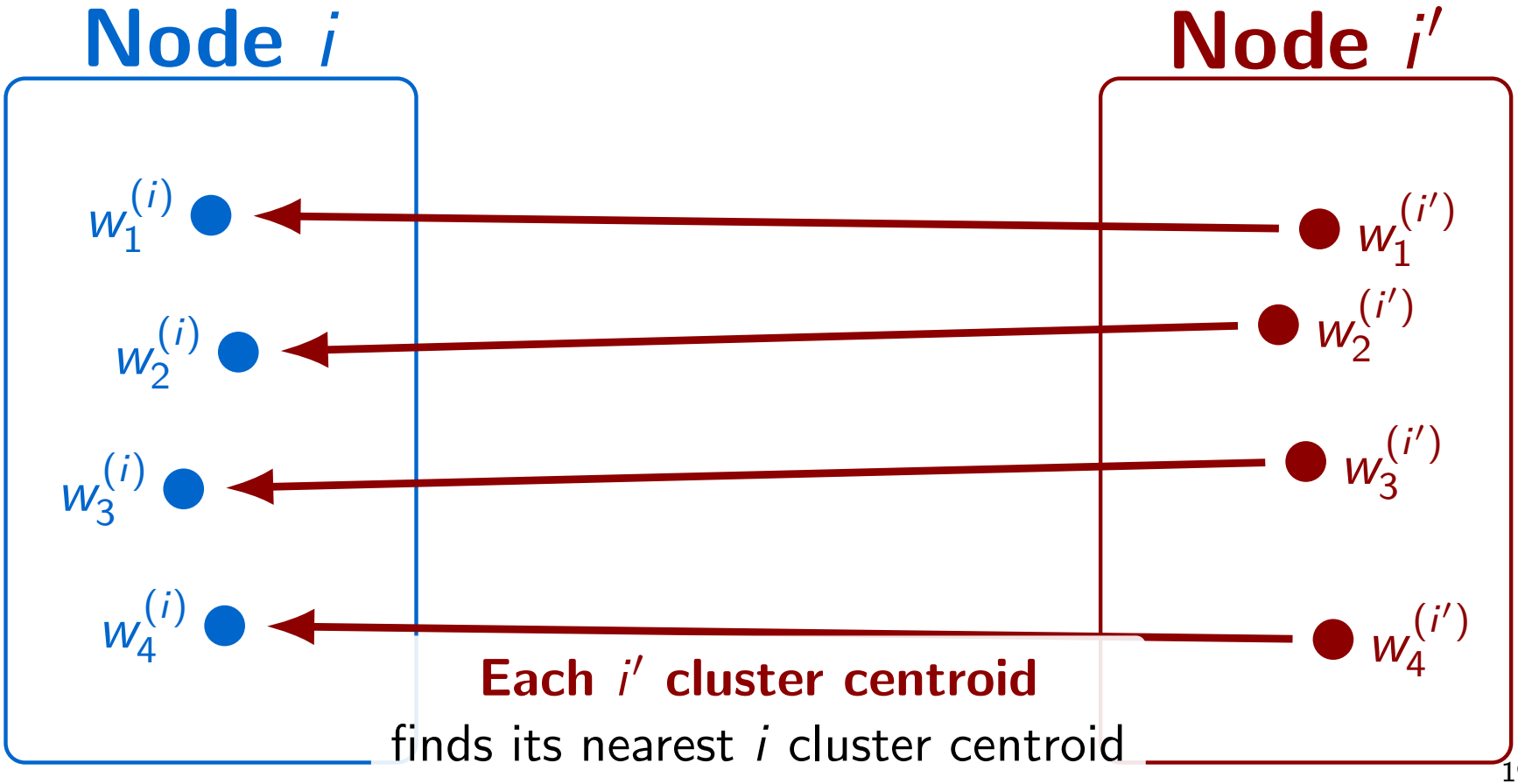
Discrepancy term I: closest i' cluster centroid to each i cluster centroid (Symmetric scenario)

$$\sum_{c=1}^{k} \min_{c' \in [k]} \|\mathbf{w}_c^{(i)} - \mathbf{w}_{c'}^{(i')}\|^2$$



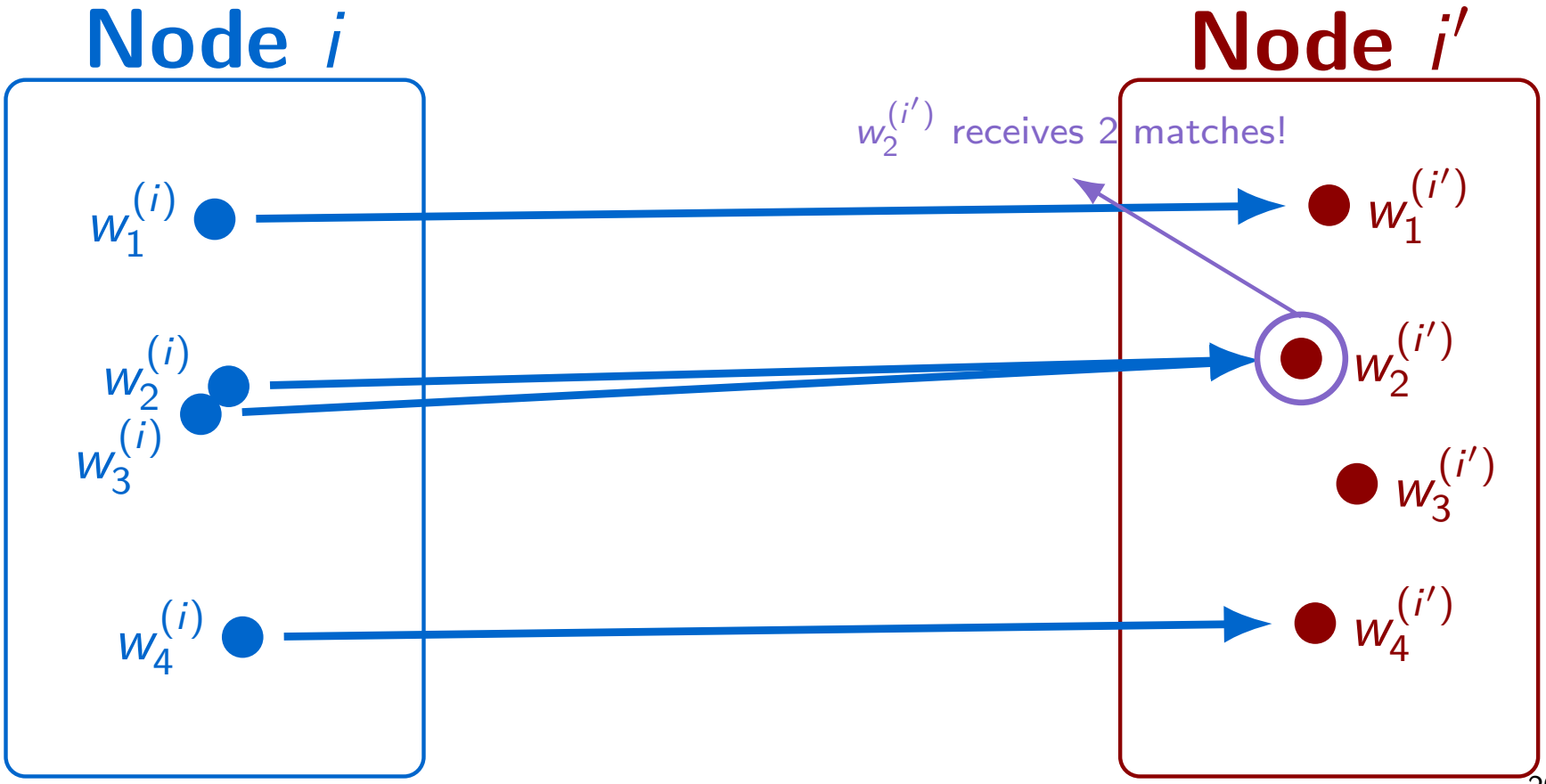
Discrepancy term II: closest i cluster centroid to each i' cluster centroid (Symmetric scenario)

$$\sum_{c=1}^{k} \min_{c' \in [k]} \|\mathbf{w}_c^{(i')} - \mathbf{w}_{c'}^{(i)}\|^2$$



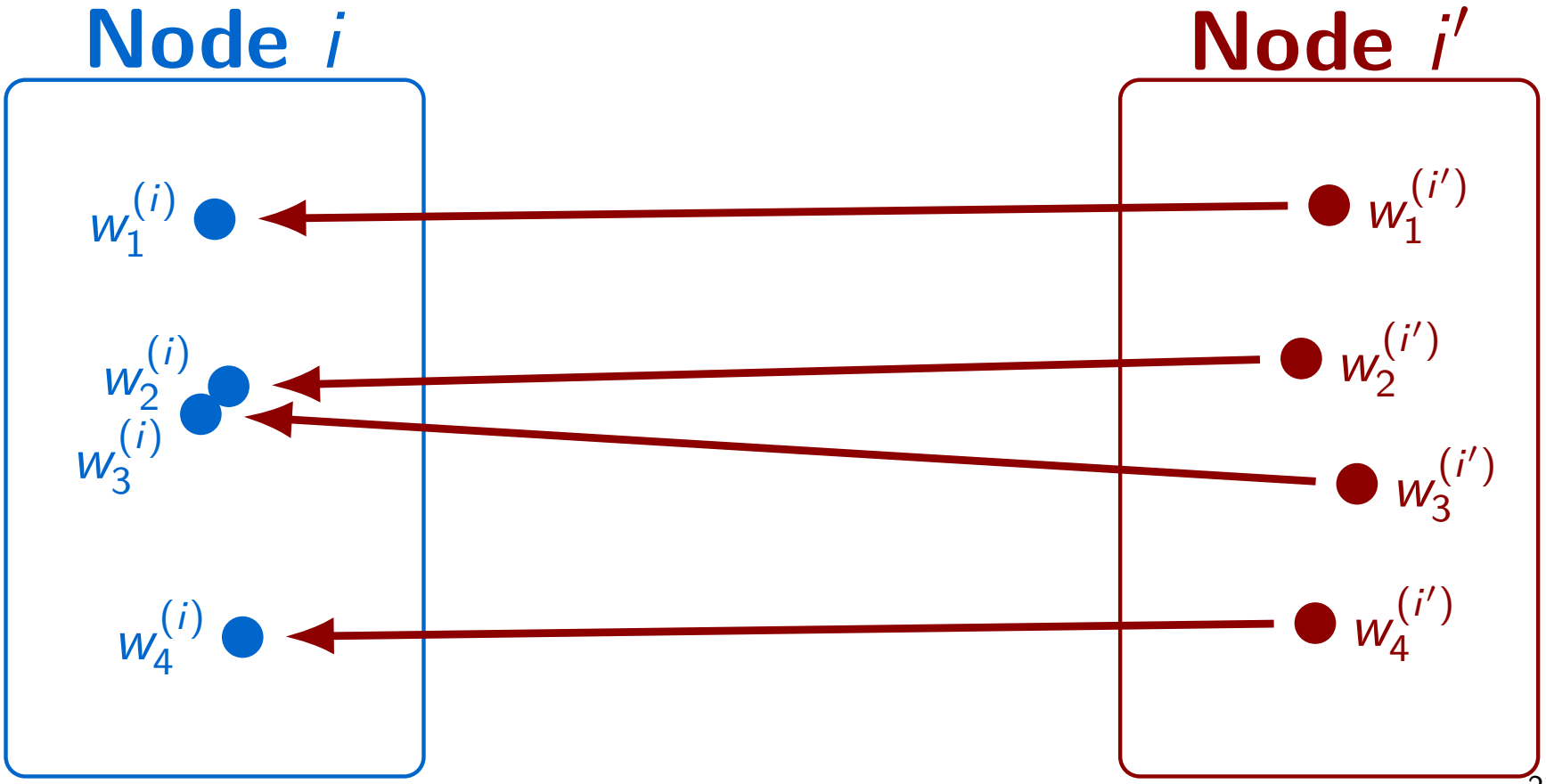
Discrepancy term I: closest i' cluster centroid to each i cluster centroid (Asymmetric scenario)

$$\sum_{c=1}^{k} \min_{c' \in [k]} \|\mathbf{w}_c^{(i)} - \mathbf{w}_{c'}^{(i')}\|^2$$



Discrepancy term II: closest i cluster centroid to each i' cluster centroid (Asymmetric scenario)

$$\sum_{c=1}^{k} \min_{c' \in [k]} \|\mathbf{w}_c^{(i')} - \mathbf{w}_{c'}^{(i)}\|^2$$



Solving GTVmin via block-wise optimization

- Iterative method with iterates $\{\mathbf{w}^{(i,t)}\}_{i=1}^{n}$.
- At iteration t, **every node updates its own block** using neighbor blocks from the previous iterate:

$$\mathbf{w}^{(i,t+1)} \in \arg\min_{\mathbf{w}^{(i,t)}} \ \ell_i(\mathbf{w}^{(i,t)}) + \lambda \sum_{i' \in \mathcal{N}^{(i)}} d^{(\mathbf{w}^{(i,t)}, \mathbf{w}^{(i',t)})}$$

- Updates can be executed **in parallel** across nodes.

Block subproblem at node i

- Local k-means loss:

$$\ell_i(\mathbf{w}^{(i)}) = \sum_{\mathbf{x} \in \mathcal{D}^{(i)}} \min_{c \in [k]} \|\mathbf{x} - \mathbf{w}^{(i,c)}\|^2.$$

- neighbor coupling uses the current neighbor cluster centroid $\{\mathbf{w}^{(i',t)}\}_{i' \in \mathcal{N}^{(i)}}$:

$$\alpha \sum_{i' \in \mathcal{N}^{(i)}} d^{(\mathbf{w}^{(i)}, \mathbf{w}^{(i',t)})}.$$

- Node i needs only: $\mathcal{D}^{(i)}$ and $\{\mathbf{w}^{(i',t)} : i' \in \mathcal{N}^{(i)}\}$.

Approximation via augmented local dataset

- The block-wise subproblem resembles k-meanswith additional terms attracting cluster centroid toward neighbors.
- Practical approximation: apply standard k-means to an **augmented local dataset**

$$\mathcal{D}^{(i,\text{aug})} = \mathcal{D}^{(i)} \cup \bigcup_{i' \in \mathcal{N}^{(i)}} \{\mathbf{m}_{i'}^{(1)}, \ldots, \mathbf{m}_{i'}^{(k)}\}.$$

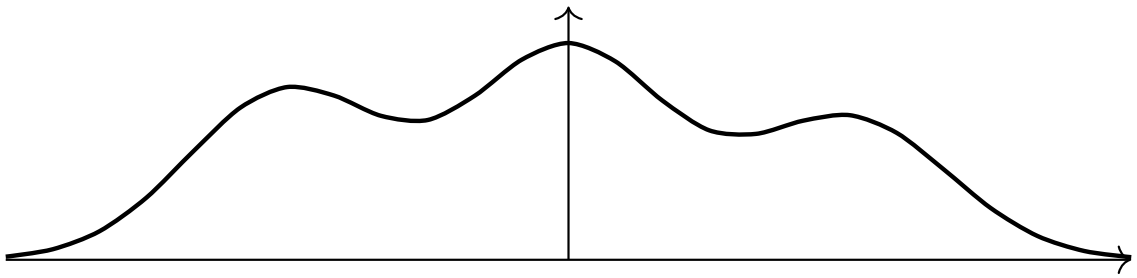
- iteration at node i: **(i)** receive neighbor cluster centroid , **(ii)** run k-means on $\mathcal{D}^{(i,\text{aug})}$, **(iii)** broadcast updated cluster centroid.

Table of Contents

1. Clustering
2. Hard clustering with k-means
3. Federated k-means
4. Soft clustering with Gaussian mixture models
5. Federated GMM

Mixture Density

Observation: The data distribution $p(\mathbf{x})$ is multi-modal



Modeling idea:

Assume $p(\mathbf{x})$ arises from $k = 3$ underlying clusters

Introducing Latent Variable

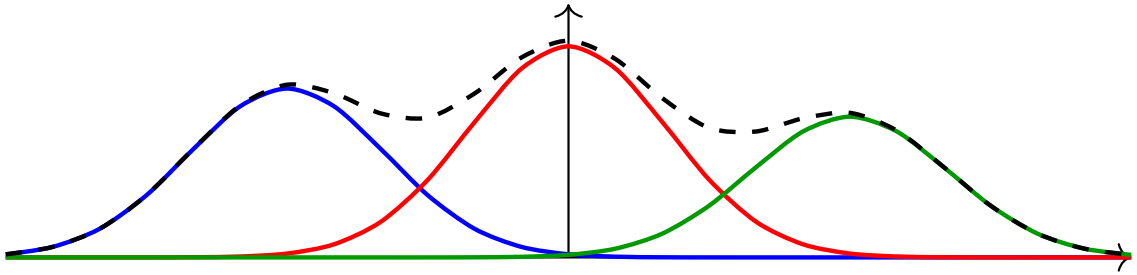
Introduce hidden variable z indicating cluster membership

$$z = (z^{(1)}, z^{(2)}, z^{(3)}), \quad z^{(c)} \in \{0, 1\}, \quad \sum_{c=1}^{3} z^{(c)} = 1$$

Possible values:

$$(1, 0, 0), \quad (0, 1, 0), \quad (0, 0, 1)$$

- $z^{(c)} = 1$ means $\mathbf{x}$ was generated by cluster c
- z is not observed (latent)



$$z \in \{(1,0,0),(0,1,0),(0,0,1)\}$$

$$p(\mathbf{x} \mid z^{(c)} = 1) = \mathcal{N}(\mathbf{x} \mid \mathbf{m}^{(c)}, \mathbf{C}^{(c)})$$

Generative Model

Step 1: Sample component

$$p(z^{(c)}) = \pi^{(c)}$$

$$p(z) = \prod_{c=1}^{k} \left(\pi^{(c)}\right)^{z^{(c)}}, \quad 0 \leq \pi^{(c)} \leq 1, \quad \sum_{c=1}^{k} \pi^{(c)} = 1$$

Step 2: Sample data given component

$$p(\mathbf{x} \mid z^{(c)} = 1) = \mathcal{N}(\mathbf{x} \mid \mathbf{m}^{(c)}, \mathbf{C}^{(c)})$$

$$p(\mathbf{x} \mid z) = \prod_{c=1}^{k} \mathcal{N}\left(\mathbf{x} \mid \mathbf{m}^{(c)}, \mathbf{C}^{(c)}\right)^{z^{(c)}}$$

Latent variable not observed $\Rightarrow$ marginalize z:

$$p(\mathbf{x}) = \sum_z p(\mathbf{x}, z) = \sum_z p(z)\, p(\mathbf{x} \mid z)$$

Marginal distribution:

$$\boxed{p(\mathbf{x}) = \sum_{c=1}^{k} \pi^{(c)}\, \mathcal{N}\big(\mathbf{x} \mid \mathbf{m}^{(c)}, \mathbf{C}^{(c)}\big)}$$

- Mixture of Gaussians
- Each component contributes proportionally via $\pi^{(c)}$

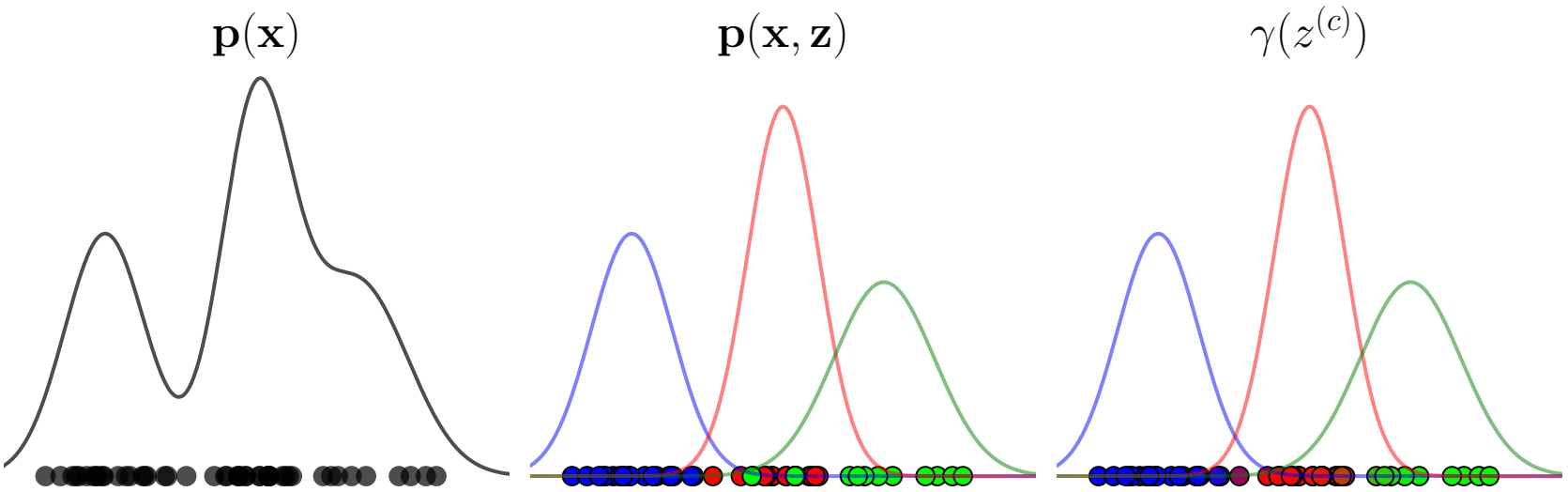
Posterior over Latent Variables

Given observation $\mathbf{x}$, compute responsibilities (soft assignments):

$$\gamma(z^{(c)}) = p(z^{(c)} = 1 \mid \mathbf{x})$$

$$= \frac{\pi^{(c)} \mathcal{N}(\mathbf{x} \mid \mathbf{m}^{(c)}, \mathbf{C}^{(c)})}{\sum_{c'=1}^{k} \pi^{(c')} \mathcal{N}(\mathbf{x} \mid \mathbf{m}^{(c')}, \mathbf{C}^{(c')})}$$

- $\pi^{(c)}$ - prior probability
- $\gamma(z^{(c)})$ - posterior probability



- **Marginal data probability** $p(\mathbf{x})$: observed data ignoring cluster assignments.
- **Complete data** $p(\mathbf{x}, z)$: shows data assigned to specific clusters (hard assignments).
- **Responsibilities** $\gamma(z^{(c)})$: soft assignment; probability each point belongs to each cluster, with colors blended according to responsibilities.

A Gaussian mixture model (GMM) with k components:

$$p(\mathbf{x}^{(r)}; \mathbf{w}) = \sum_{c=1}^{k} \pi^{(c)} \mathcal{N}\left(\mathbf{x}^{(r)}; \mathbf{m}^{(c)}, \mathbf{C}^{(c)}\right)$$

- $\pi^{(c)}$: mixture weight, $\pi^{(c)} \geq 0$, $\sum_{c=1}^{k} \pi^{(c)} = 1$
- $\mathbf{m}^{(c)}$: mean of component c
- $\mathbf{C}^{(c)}$: covariance of component c
- $\mathbf{w}^{(c)} = \{\pi^{(c)}, \mathbf{m}^{(c)}, \mathbf{C}^{(c)}\}$
- $\mathbf{w} = \{\mathbf{w}^{(c)}\}_{c=1}^{k}$

Maximizing Log-Likelihood

- **Learning objective:** choose parameters $\mathbf{w}$ to maximize log-likelihood of dataset

$$L(\mathbf{w}) = \log p(\mathcal{D}; \mathbf{w}) = \sum_{r=1}^{m} \log p(\mathbf{x}^{(r)})$$

- **Key difficulty:** log of a sum over latent components

$$\log \sum_{c=1}^{k} \pi^{(c)} \mathcal{N}(\mathbf{x}^{(n)} \mid \mathbf{m}^{(c)}, \mathbf{C}^{(c)})$$

- **Solution:** expectation–maximization (EM) algorithm; latent cluster assignments make direct maximization hard; EM iteratively estimates assignments and parameter

EM algorithm

Iteratively estimate hidden assignments (E-step) and maximize parameters (M-step).

1. **E-Step:** Compute responsibilities:

$$\gamma_t^{(r,c)} = \frac{\pi_t^{(c)} \mathcal{N}\left(\mathbf{x}^{(r)}; \mathbf{m}_t^{(c)}, \mathbf{C}_t^{(c)}\right)}{\sum_{c'=1}^{k} \pi_t^{(c')} \mathcal{N}\left(\mathbf{x}^{(r)}; \mathbf{m}_t^{(c')}, \mathbf{C}_t^{(c')}\right)}$$

EM algorithm

Iteratively estimate hidden assignments (E-step) and maximize parameters (M-step).

1. **E-Step:** Compute responsibilities $\gamma_t^{(r,c)}$
2. **M-Step:** Update parameters using responsibilities:

$$\pi_{t+1}^{(c)} = \frac{1}{N} \sum_{r=1}^{N} \gamma_t^{(r,c)}$$

$$\mathbf{m}_{t+1}^{(c)} = \frac{\sum_r \gamma_t^{(r,c)} \mathbf{x}^{(r)}}{\sum_r \gamma_t^{(r,c)}}$$

$$\mathbf{C}_{t+1}^{(c)} = \frac{\sum_r \gamma_t^{(r,c)} (\mathbf{x}^{(r)} - \mathbf{m}_{t+1}^{(c)})(\mathbf{x}^{(r)} - \mathbf{m}_{t+1}^{(c)})^\top}{\sum_r \gamma_t^{(r,c)}}$$

1. Initialize $\pi^{(c)}$, $\mathbf{m}^{(c)}$, $\mathbf{C}^{(c)}$

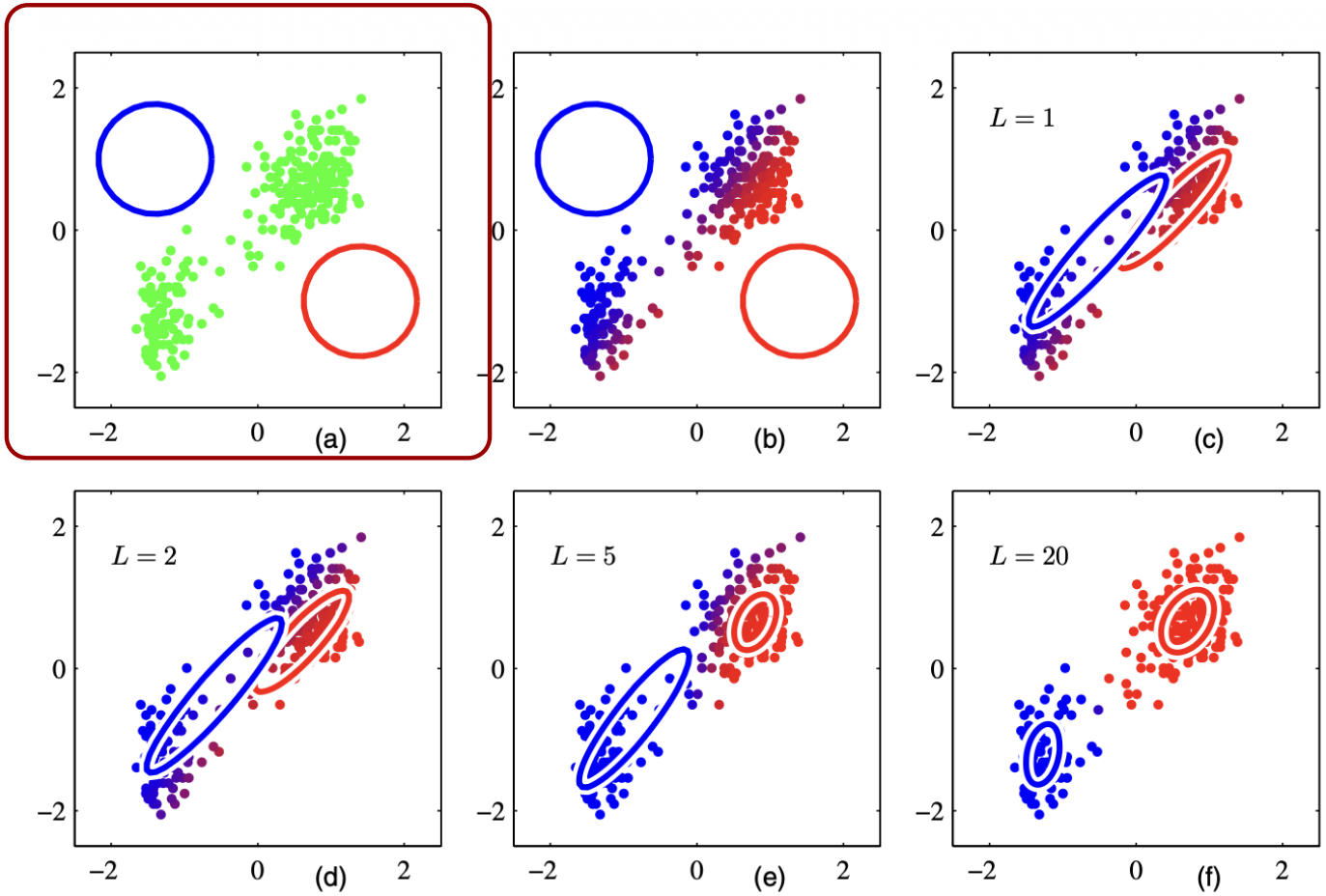


image source:

Bishop's "Pattern recognition ..."

2. **E-Step:** Compute responsibilities $\gamma_t^{(r,c)}$

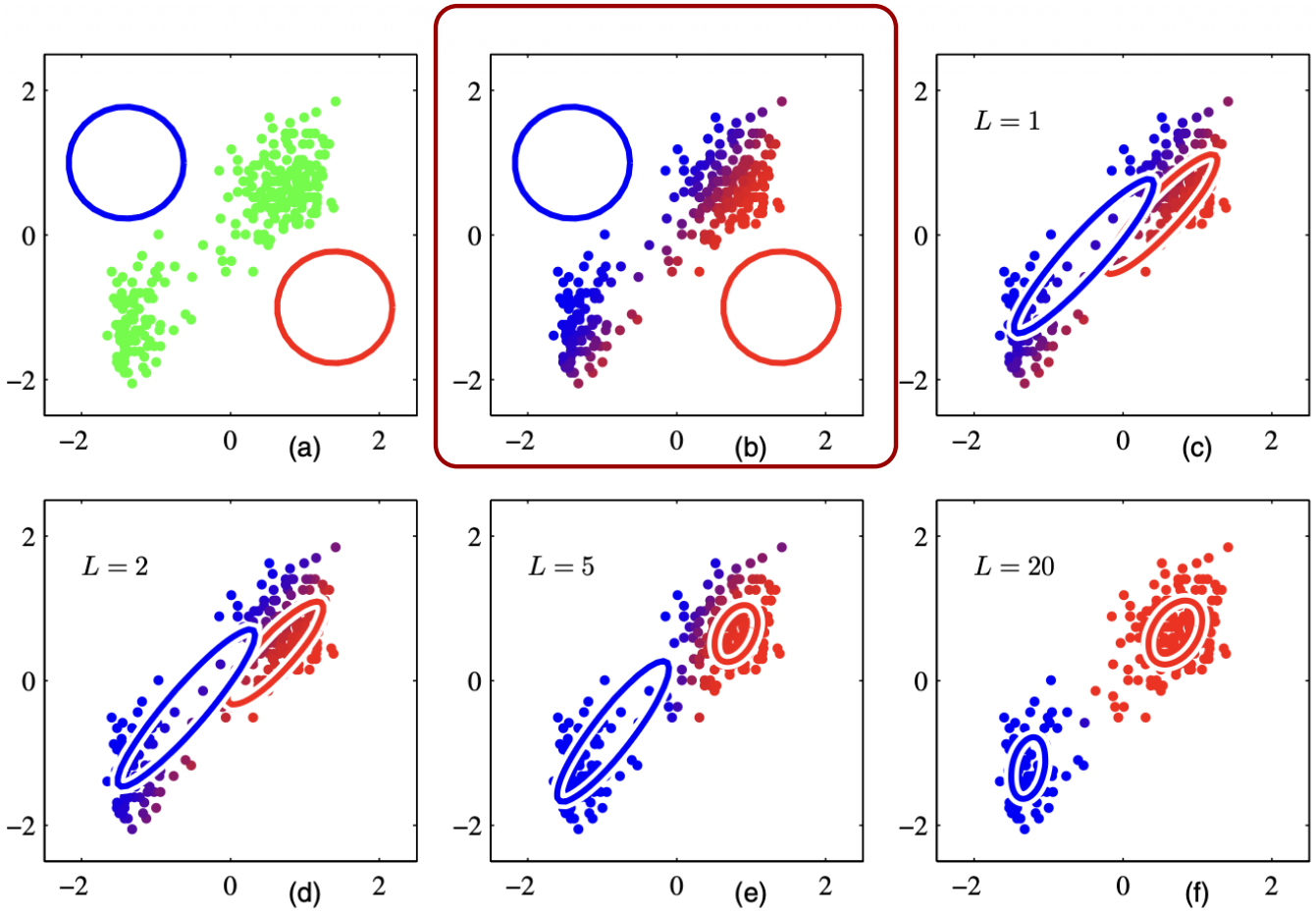


image source: Bishop's "Pattern recognition ..."

3. **M-Step:** Update $\pi^{(c)}$, $\mathbf{m}^{(c)}$, $\mathbf{C}^{(c)}$

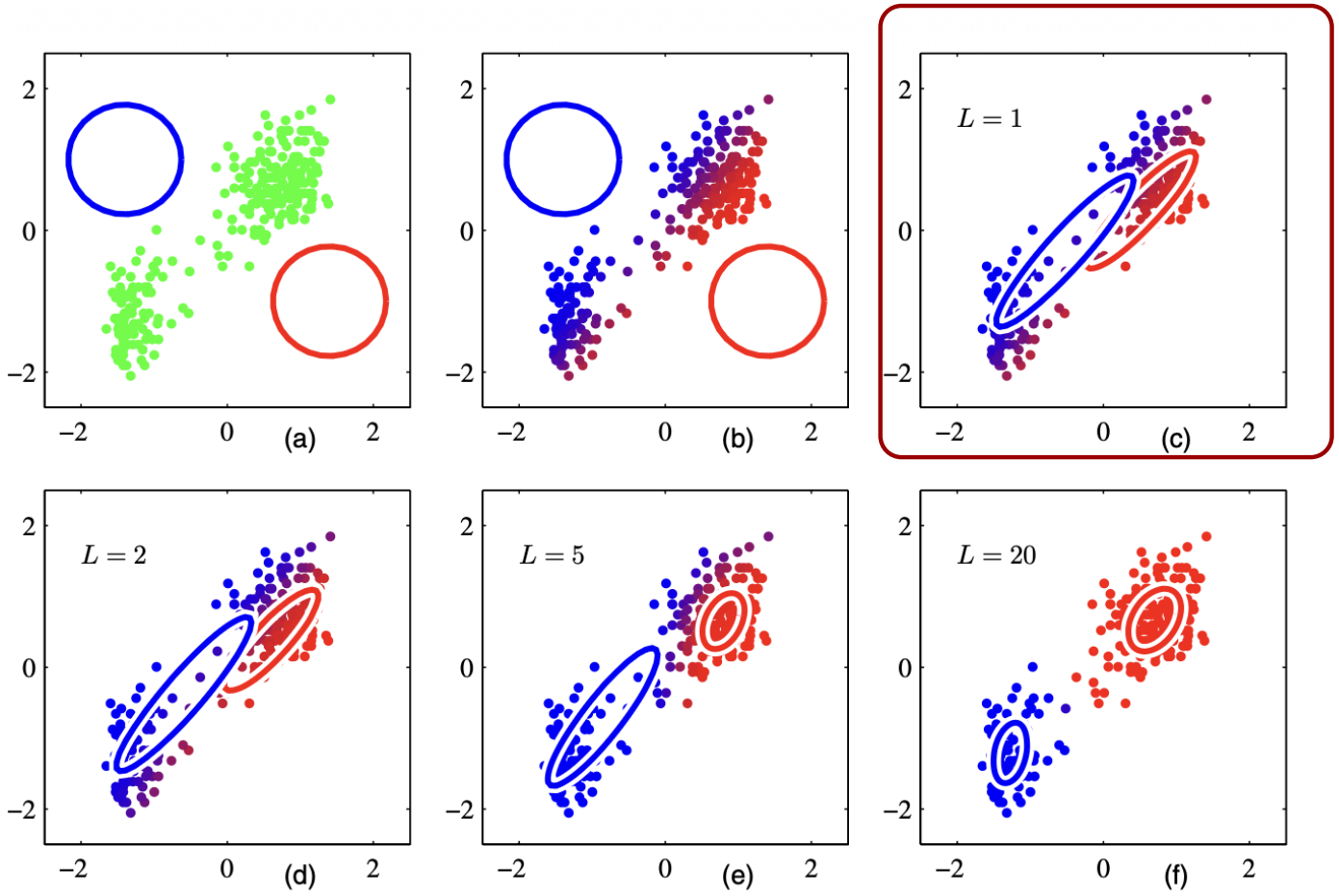


image source: Bishop's "Pattern recognition ..."

Repeat E-Step and M-step until convergence:

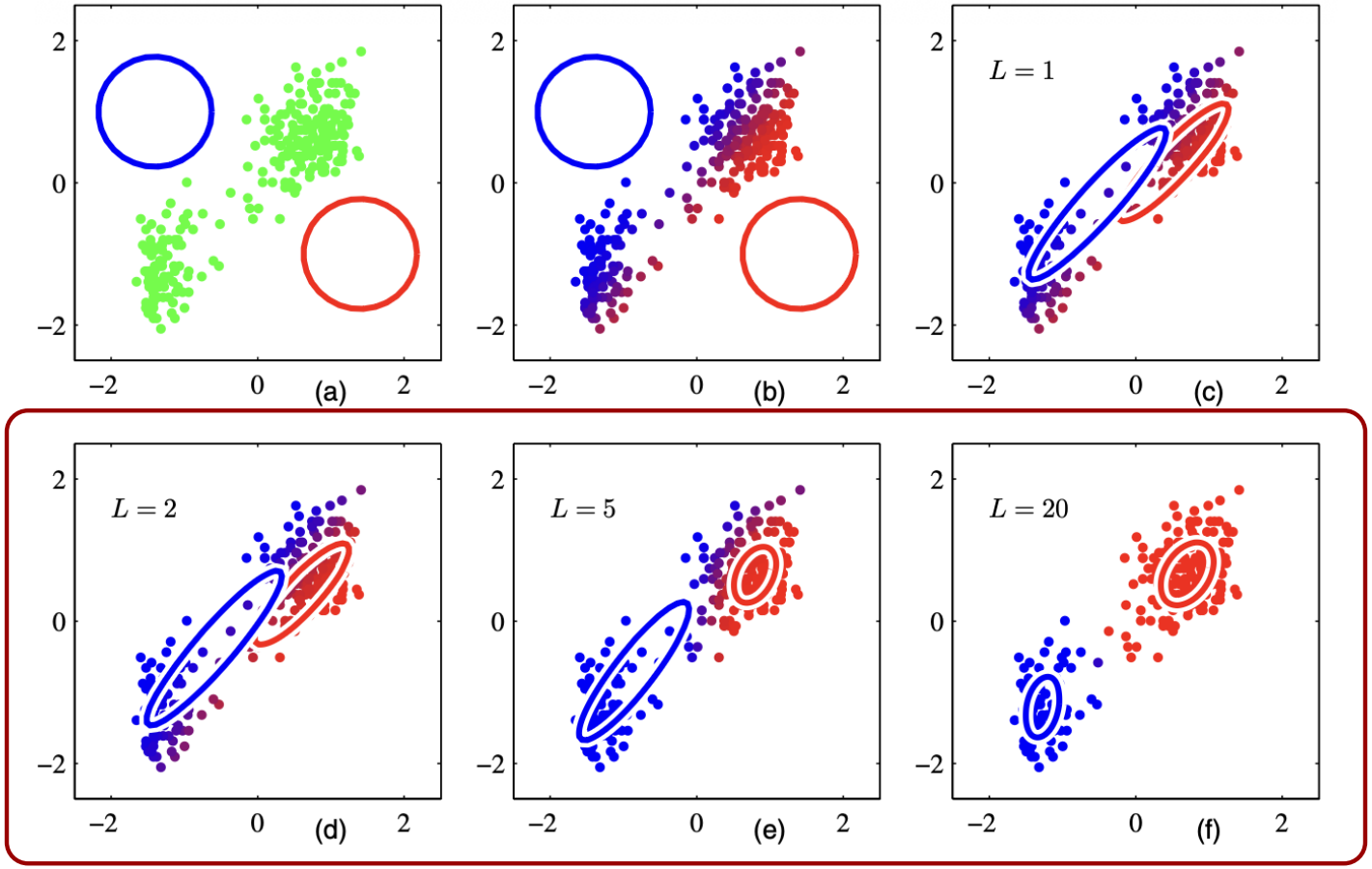


image source: Bishop's "Pattern recognition ..."

Limitations of EM

1. Requires full dataset each iteration
2. M-step requires closed-form updates (limits model flexibility)
3. Not suitable for:
   - Large-scale datasets
   - Distributed/federated settings
   - Models without analytic updates

Gradient EM: Motivation

- Replace closed-form M-step with gradient updates
- Useful when:
  - M-step is intractable
  - Data is distributed across clients
  - We want to include regularization $\mathcal{R}(\mathbf{w})$ on parameter
- Gradient ascent on regularized log-likelihood:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \nabla_{\mathbf{w}} \big(L(\mathbf{w}) + \mathcal{R}(\mathbf{w}) \big)$$

- Enables:
  - Mini-batch learning
  - Federated optimization
  - Incorporation of priors or penalties via $\mathcal{R}(\mathbf{w})$

Gradient EM: Implementation

- **E-step:** compute responsibilities

$$\gamma^{(c,r)} = p(z^{(r)} = c \mid \mathbf{x}^{(r)}, \mathbf{w}^{\text{old}})$$

- **M-step:** perform gradient ascent on regularized log-likelihood:

$$\sum_r \sum_c \gamma^{(c,r)} \log p(\mathbf{x}^{(r)}, z^{(r)} = c; \mathbf{w}) + \mathcal{R}(\mathbf{w})$$

- Update parameter :

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \nabla_{\mathbf{w}} \big(L(\mathbf{w}) + \mathcal{R}(\mathbf{w}) \big)$$

Table of Contents

1. Clustering
2. Hard clustering with k-means
3. Federated k-means
4. Soft clustering with Gaussian mixture models
5. Federated GMM

From Regularized Log-Likelihood to GTVMin

$$\underbrace{L(\mathbf{w})}_{\text{Log-Likelihood}} + \underbrace{\mathcal{R}(\mathbf{w})}_{\text{reg.term}}$$

In Federated Learning:

- $\mathcal{R}(\mathbf{w}^{(i)}) = \alpha \sum_{i' \in \mathcal{N}^{(i)}} D\left(\mathbf{w}^{(i)}, \mathbf{w}^{(i')}\right)$
- $D(\cdot, \cdot)$ can be any divergence or distance (e.g., Euclidean, KL) measuring parameters/ distribution similarity.
- Encourages consistent models across nodes without sharing raw data.
- Update remains local, requiring only neighbor parameters.

Regularized Log-Likelihood is an instance of GTVMin

Discrepancy / Variation Measure for GTVMin

- Use **symmetrized KL divergence** (SKL) between node distributions:

$$D_{\mathrm{SKL}}\Big(p(\cdot;\mathbf{w}^{(i)}), p(\cdot;\mathbf{w}^{(i')})\Big) = \frac{1}{2}\Big(\mathrm{KL}\Big(p(\cdot;\mathbf{w}^{(i)}) \,\|\, p(\cdot;\mathbf{w}^{(i')})\Big) + \mathrm{KL}\Big(p(\cdot;\mathbf{w}^{(i')}) \,\|\, p(\cdot;\mathbf{w}^{(i)})\Big)\Big)$$

- Captures variation of GMM across neighboring nodes
- Governs smoothness / similarity of local models

SKL–GTVMin

- Federated learning over an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
- Nodes hold private datasets $\mathcal{D}^{(i)}$ and and local GMM parameters $\mathbf{w}^{(i)}$
- Goal: minimize **SKL–GTVMin objective**:

$$\sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{(i,i')\in\mathcal{E}} D_{\mathrm{SKL}}\Big(p(\cdot;\mathbf{w}^{(i)}), p(\cdot;\mathbf{w}^{(i')})\Big)$$

where

$$L_i\left(\mathbf{w}^{(i)}\right) = -\frac{1}{m_i} \sum_{\mathbf{x}\in\mathcal{D}^{(i)}} \log p(\mathbf{x};\mathbf{w}^{(i)})$$

Block-Wise Optimization for $\mathbf{w}^{(i)}$

- Optimize each node's parameters individually while holding neighbors fixed

$$\mathbf{w}^{(i,t+1)} = \arg\min_{\mathbf{w}^{(i)}} L_i\left(\mathbf{w}^{(i)}\right) \tag{1}$$

$$+ \alpha \sum_{i' \in \mathcal{N}^{(i)}} D_{\mathrm{SKL}}\Big(p(\cdot; \mathbf{w}^{(i)}), p(\cdot; \mathbf{w}^{(i',t)})\Big) \tag{2}$$

- neighbor models p_j^t are fixed
- Forward KL is majorized using fixed self-sample p_i^t:

$$\mathbb{E}_{\mathbf{x} \sim p(\cdot; \mathbf{w}^{(i,t)})}\left[\log p(\mathbf{x}; \mathbf{w}^{(i)})\right]$$

Objective at Node i

$$\mathcal{L}_i^{\text{sur}} = \underbrace{\mathbb{E}_{\mathbf{x}\in\mathcal{D}^{(i)}}[-\log p(\mathbf{x};\mathbf{w}^{(i)})]}_{\text{local data}} + \underbrace{\sum_{i'\in\mathcal{N}^{(i)}} \mathbb{E}_{\mathbf{x}\sim p(\cdot;\mathbf{w}^{(i',t)})}[-\log p(\mathbf{x};\mathbf{w}^{(i)})]}_{\text{neighbor cross-entropy}} + \underbrace{\sum_{i'\in\mathcal{N}^{(i)}} \mathbb{E}_{\mathbf{x}\sim p(\cdot;\mathbf{w}^{(i,t)})}[\log p(\mathbf{x};\mathbf{w}^{(i)})]}_{\text{"self" cross-entropy}}$$

- First term: encourages the local model to fit local dataset.
- Second term: encourages agreement with neighbor samples.
- Third term: prevent overfit to p_i^t

Algorithm Summary (Update on node i)

1. Generate self-samples $\mathbf{x} \sim p(\cdot; \mathbf{w}^{(i,t)})$
2. Gather neighbor samples $\mathbf{x} \sim p(\cdot; \mathbf{w}^{(i',t)}), \quad i' \in \mathcal{N}^{(i)}$
3. Create augmented dataset $\mathcal{D}^{(i,\text{aug})}$ from local data + self-samples + neighbor samples
4. Construct surrogate loss $\mathcal{L}_i^{\text{sur}}\big(\mathbf{w}^{(i)}\big)$
5. Perform gradient-based optimization (Adam) w.r.t $\mathbf{w}^{(i)}$
6. Update $\mathbf{w}^{(i,t+1)}$

Augmented Local dataset Approximation

- Approximate block-wise optimization using augmented dataset:

$$\mathcal{D}^{(i,\text{aug})} = \mathcal{D}^{(i)} \cup \{\text{samples from} p(\cdot; \mathbf{w}^{(i',t)}\}$$

- Forward KL term $\mathbb{E}_{\mathbf{x}\sim p(\cdot;\mathbf{w}^{(i,t)})}\left[\log p(\mathbf{x}; \mathbf{w}^{(i)})\right]$ is dropped.

$\rightarrow$ simpler augmented-dataset MLE

- Provides practical decentralized solver:
  - No raw data sharing (only parameters)
  - Can be implemented with `sklearn.mixture.GMM`

Wrap-Up

- **Federated clustering**:
  - Local datasets across nodes; communication via a neighbor graph.
  - Achieves global consistency without sharing raw data.
  - Uses GTVMin to align local models.
- **Block-wise optimization**:
  - Nodes update local models using neighbor information.
  - Iterative and parallel.
- **Augmented local dataset**:
  - Used to approximate the local objective.
  - Easily implemented with high-level libraries.

Bonus: Using MPI for Federated learning

Repo with code: https://github.com/shamPJ/MPI-P2P

- **MPI (Message Passing Interface)**: message-passing standard for HPC parallel programming.
- `comm = MPI.COMM_WORLD`:
  - Represents all processes in the job.
  - Each process has a unique rank (0 to N-1).
  - Provides methods: send, recv, broadcast, gather, scatter.
- Processes communicate by sending messages, e.g., ‘comm.send(data, dest=1)‘.

MPI Key Points

- All processes share the `comm` context.
- Each rank executes code independently but can synchronize via communication.
- Collective operations (broadcast, gather) operate over `comm`.

Federated Learning Setup

- Peer-to-peer FL using `mpi4py`.
- Each rank maintains a local Gaussian Mixture Model (GMM).
- Ranks exchange parameters with neighbors and augment datasets.
- Rank 0 computes centralized baseline for comparison.

MPI on Aalto HPC Triton

- OpenMPI + `mpi4py`
- Launch command:

```
srun mpi_p2p_gmm.py
```

MPI Operations Used

`comm.bcast()`	Broadcast
`comm.isend()`	Non-blocking send
`comm.irecv()`	Non-blocking receive
`MPI.Request.Waitall()`	Wait for all sends
`comm.Barrier()`	Synchronize ranks
`comm.gather()`	Gather information

- Each `isend` must match an `irecv` to avoid deadlocks.
- MPI is CPU/GPU agnostic:
  - Use CUDA-aware MPI for GPU tensors.
  - Otherwise, transfer to CPU before sending.
- `Barrier` ensures consistent iteration steps and outputs.